

RESPUESTAS TEÓRICAS M2C3

Tipos de Datos en Python:

- **Números**
 - Enteros. 3
 - Punto flotante. Incluyen decimales. P. ej: 3.14
 - Complejos. Incluyen los números imaginarios
- **Cadena de caracteres (strings)**
 - Secuencias de caracteres encerradas entre comillas simples (') o dobles ("). Por ejemplo: 'Hola', "Python".
 - Las cadenas de caracteres en Python son inmutables, lo que significa que no se pueden modificar una vez creadas.
- **Listas**
 - Colecciones ordenadas y modificables de elementos, separados por comas y encerrados entre corchetes ([]). Por ejemplo: [1, 2, 3], ['a', 'b', 'c'].
 - Los elementos de una lista pueden ser de cualquier tipo de dato, incluidas otras listas.
- **Tuplas**
 - Colecciones ordenadas e inmutables de elementos, separados por comas y encerrados entre paréntesis (). Por ejemplo: (1, 2, 3), ('a', 'b', 'c').
 - Aunque los elementos de una tupla no se pueden modificar, sí se pueden acceder mediante su índice.
- **Diccionarios**
 - Colecciones desordenadas de pares clave-valor, separados por comas y encerrados entre llaves {}. Por ejemplo: {'nombre': 'Juan', 'edad': 30}.
 - Los elementos de un diccionario se acceden mediante sus claves, no por índices, y las claves deben ser únicas e inmutables (como cadenas, números o tuplas).

- Booleanos (True, False)
 - Representan dos valores: Verdadero (True) y Falso (False). Se utilizan en expresiones lógicas y de control de flujo, como condicionales y bucles.
- Ninguno (None)
 - Fija la ausencia de valor o la falta de definición.
- Bytes o byte arrays
 - Almacenamiento de datos binarios como archivos, imágenes o datos.

Convención de Nomenclatura para Variables en Python:

- Utiliza nombres descriptivos que indiquen el propósito de la variable.
- Utiliza minúsculas para los nombres de las variables, con un guión bajo para separar palabras.
- Evita utilizar palabras reservadas de Python como nombres de variables.

Heredoc en Python:

Son cadenas de texto multilínea, las cuales se pueden definir utilizando comillas triples (''' o '''). Estas cadenas de texto pueden abarcar múltiples líneas de código sin necesidad de utilizar caracteres de escape como \n. La indentación se mantiene tal como se escribe, lo que permite una mejor legibilidad y mantenimiento del código.

Interpolación de Cadenas:

La interpolación de cadenas es una técnica utilizada para construir strings de texto dinámicas que incluyen valores de variables dentro de ellas. En Python, hay varias formas de realizar interpolación de cadenas, aunque la más extendida es mediante la función f o f-strings.

Las f-strings se definen precediendo la cadena con la letra f y pueden incluir expresiones dentro de corchetes ({ }) que serán convertidas en cadenas automáticamente.

Uso de comentarios en Python:

Usar comentarios en Python es una práctica fundamental para mejorar la legibilidad y comprensión del código. Puede utilizarse en ciertas situaciones (mediante #):

- Explicar una sección del código
- Explicación de funciones y métodos
- Aclarar código complejo o poco intuitivo
- Desactivar temporalmente código

Sin embargo, un uso excesivo de los comentarios o no actualizarlos junto con el código puede llevar a ciertos errores de comprensión por parte de otros desarrolladores. Es más conveniente realizar una descripción detallada mediante los nombres de las variables y funciones facilitando la comprensión sin la necesidad de añadir más líneas de comentarios.

Diferencias entre Aplicaciones Monolíticas y de Microservicios:

Las aplicaciones monolíticas son aquellas en las que todo el código está incluido dentro de una sola unidad, mientras que en una de microservicios, la aplicación se divide en sistemas más pequeños e independientes que se comunican entre sí a través de API. En las aplicaciones monolíticas, los cambios y actualizaciones pueden ser más difíciles de implementar debido a la interdependencia de los distintos componentes, mientras que en las aplicaciones de microservicios, cada servicio puede ser tratado independientemente.

A su vez, las aplicaciones monolíticas podrían llegar a ser más simples de desarrollar inicialmente, ya que requieren una cantidad menor de componentes externos siendo el código más próximo y cercano. Por otro lado, las aplicaciones de microservicios, pueden ofrecer mayor flexibilidad y escalabilidad a largo plazo, aunque requieren de herramientas de comunicación entre los distintos servicios.